CrewCut QAOA: One Qubit per Repair against Standard QAOA for Two-Crew Restoration Scheduling

RestoreBench-Q team, 11 October 2026. Working name. Every number below is read from a result file in this repository; Section 8 maps each claim to its file.

Abstract

After a storm, utilities decide which of two crews repairs each damaged line. We encode this decision for QAOA with one qubit per repair and a pair cost derived from the scheduling model (CrewCut). We compare that encoding with standard QAOA, which uses one qubit per repair and crew plus a penalty, and with the dispatch rules utilities use today. On the IEEE 13 node feeder, the best of 100 depth 1 CrewCut shots is 0.32% above the best assignment. The published conversion algorithm is 1.47% above it on the same 20 instances, and the FirstEnergy and Edison Electric Institute rules are 19.4% and 14.2% above it on 300 instances. Against standard QAOA, with the same trainer, depth and number of starts, CrewCut returns a best plan 3.1 to 4.6 times as often at depth 2. It does so on half the qubits and fewer than half the pair gates, and the factor grows from 6 to 10 repairs. Under IonQ's documented Aria 1 noise model, dropping half the pair terms raises the share of the ideal gain kept from 45% to 52%. Exact classical search on the same fitted cost still reaches the optimum more often than QAOA at these sizes (30 of 30 instances against 16 of 30). We claim no quantum advantage.

1. Introduction

Crew assignment after a storm is a scheduling problem. Each damaged line j has repair hours p_j and priority weight w_j. Each crew works through its repairs in decreasing order of urgency $r_j = w_j / p_j$, and the cost is the priority-weighted time without power (Tan et al. [1]). In practice the order is set by dispatch rules: largest load first (FirstEnergy) or largest load per hour (Edison Electric Institute).

The standard way to give such a problem to QAOA [2] is one binary variable per repair and crew, with a penalty that forces each repair onto exactly one crew [3, §6.3]. For two crews that costs $2n$ qubits and n^2 pair terms. Most of the circuit then encodes the constraint, not the cost.

Contribution. We show that for two crews the cost is exactly a pair cost on one qubit per repair, with no penalty and no invalid bitstrings. We measure, on the same instances and with the same optimiser, what this changes:

1. qubits and two-qubit gates;
2. the chance that one shot is a best plan;
3. the share of the gain over random plans that the shots keep;
4. the effect of a sparse cost layer under IonQ's noise model.

We also report every measurement in which a classical method wins.

2. Formulations

CrewCut (one qubit per repair). Write $s_j = +1$ for crew A and -1 for crew B. With each crew following Smith's rule, two repairs on the same crew add

$$J_{ij} = p_i \, p_j \, \min(r_i, r_j),$$

so that $\text{cost}(s) = \text{constant} + \tfrac{1}{2} \sum_{i<j} J_{ij} \, s_i \, s_j$ [4, §6]. Every bitstring is a valid plan. For n repairs the circuit has n qubits and $n(n-1)/2$ pair terms per cost layer.

Standard QAOA (one qubit per repair and crew). We use one binary variable x_{jc} per repair j and crew c [4, §2.1], with the schedule cost as single and pair terms and the penalty $\lambda \sum_j (\sum_c x_{jc} - 1)^2$ [3, §6.3]. For two crews this gives $2n$ qubits and n^2 pair terms. Bitstrings that place a repair on zero or two crews are invalid and are discarded.

We give the baseline every advantage we could. The penalty is run at 1/8, 1/4, 1/2, 1, 2 and 4 times a bound above which every lowest state is valid, and at each repair's own bound. For each instance we then report the penalty that produced the most best-plan shots. That choice uses the answer, so the baseline in the tables below is an oracle-tuned upper bound on standard QAOA.

3. Experimental setup

Item	Setting
Instances	Repair hours and weights from the instance generator of Tan et al. [1, §6]; 30 instances at 6 and 8 repairs, 10 at 10 repairs; two crews
QAOA	Depth 1 and 2; the same trainer and 6 random starts for both forms; exact state-vector distribution
Scoring	Every shot scored on the true schedule cost, not the pair cost
Statistic	Median over instances, with quartiles in the result files
Reference rate	A uniformly random valid plan is a best plan with probability $2/2^n$ (the two crews can be swapped)

Metrics.

- *Best-plan shots*: the share of shots that are a best plan.
- *Shots for 95%*: the number of shots needed for a 95% chance of at least one best plan (repetitions to target [5]).
- *Gain kept*: (random mean − shot mean) / (random mean − optimum), measured over valid shots.
- *Regret*: $(V - V) / V$.

4. Results

4.1 Against standard QAOA

Table 1. Share of shots that are a best plan, median over instances. The ratio is CrewCut's median divided by the median of oracle-tuned standard QAOA.

Repairs	Depth	CrewCut qubits / pair gates	Standard qubits / pair gates	Random valid plan	Standard, penalty at the bound	Standard, oracle-tuned	CrewCut	Ratio
6	1	6 / 15	12 / 36	3.13%	1.44%	2.94%	**7.08%**	**2.4×**
6	2	6 / 15	12 / 36	3.13%	3.13%	3.31%	**10.35%**	**3.1×**
8	1	8 / 28	16 / 64	0.78%	0.35%	0.74%	**2.29%**	**3.1×**
8	2	8 / 28	16 / 64	0.78%	0.88%	0.90%	**3.63%**	**4.0×**
10	1	10 / 45	20 / 100	0.20%	0.08%	0.18%	**0.62%**	**3.4×**
10	2	10 / 45	20 / 100	0.20%	0.20%	0.22%	**1.01%**	**4.6×**

CrewCut returns more best-plan shots than oracle-tuned standard QAOA on every instance: 30 of 30 at 6 and at 8 repairs, and 10 of 10 at 10 repairs. The ratio rises with problem size at both depths (2.4 → 3.1 → 3.4 at depth 1; 3.1 → 4.0 → 4.6 at depth 2).

Table 2. Shots needed and gain kept, median over instances.

Repairs	Depth	Shots for 95%, CrewCut	Shots for 95%, standard (oracle)	Gain kept, CrewCut	Gain kept, standard (oracle)	Valid shots, standard (oracle)
6	1	41	101	59.9%	0.2%	93.5%
6	2	27	89	73.3%	4.2%	99.2%
8	1	129	406	59.7%	0.2%	92.8%
8	2	81	330	70.9%	7.8%	99.3%
10	1	485	1,639	55.6%	0.1%	93.2%
10	2	296	1,372	68.8%	6.4%	99.2%

Table 2 shows why the baseline loses. To make nearly every shot valid, the penalty has to dominate the cost. The trained circuit then learns validity and almost none of the schedule. Its best-plan rate settles at that of a random valid plan. An independent check confirms this at 6, 8 and 10 repairs (`checks/standard_qaoa_checks.py`, largest gap 0.019 points). At lower penalties the circuit keeps more of the cost but loses validity: at the bound, only 37% to 42% of depth 1 shots are valid. CrewCut has no such trade-off, because every bitstring is a plan.

Hardware cost at 12 repairs. These counts are taken from the circuits, not simulated. One cost layer and one mixer layer take 66 two-qubit gates for CrewCut against 144 for the standard form when every pair is connected. On a heavy hex layout they take 205 against 420.

4.2 Against the dispatch rules utilities use

Table 3. IEEE 13 node feeder, 12 repairs, two crews, percent above the best assignment.

Method	Instances	Mean above the best assignment
FirstEnergy rule (largest load first)	300	19.37%
Edison Electric Institute rule (largest load per hour)	300	14.17%
Conversion algorithm (published heuristic [1])	300	1.65%
Conversion algorithm	20	1.47%
Random plans, best of 100	20	0.62%
CrewCut QAOA depth 1, best of 100 shots	20	**0.32%**
CrewCut QAOA depth 2, best of 100 shots	20	0.66%
CrewCut QAOA depth 3, best of 100 shots	20	1.13%

On 12 instances solved to proven optimality, the best assignment is 0.15% above the optimum on average. CrewCut's 0.32% is therefore measured from a reference that is itself within 0.15% of the optimum. The best CrewCut plan uses 16.2% fewer customer-hours than the FirstEnergy rule and 12.8% fewer than the Edison Electric Institute rule.

Two caveats belong with Table 3:

- **Random plans beat the heuristic too.** The best of 100 random plans is also within the conversion algorithm's 1.47%. QAOA at depth 1 halves the random plans' gap.
- **Depth does not help best-of-100 here.** On this run the best of 100 shots gets worse with depth. The chance that one shot is a best assignment does rise with depth (0.34%, 0.42%, 0.47% against 0.132% at random), but the shots spread over fewer distinct plans.

Equal budget. When the evaluations used to fit the pair cost are counted (the A1 harness, 30 instances, fit on 150 plans), QAOA at depth 1 reaches a median regret of 0.28% at 100 evaluations. At the same budget random plans reach 0.39%, and the conversion algorithm sits at 0.54%.

Growth with size and depth. On independent repairs, the chance that one shot is the best plan, as a multiple of random sampling, rises with both depth and size. From 8 to 14 repairs it goes from 2.5 to 3.1 at depth 1, from 3.7 to 4.9 at depth 2, and from 4.0 to 7.2 at depth 3.

4.3 Sparse cost layer under noise

Dropping the weakest pair terms shortens the circuit. CrewCut has no penalty terms, so every remaining term carries cost information. Table 4 scores the sparse layer on the full cost under IonQ's documented depolarising noise model [6]: one error channel after every gate, 300 error patterns per circuit, 12 repairs, 30 instances, depth 2.

Table 4. Gain over random plans kept, median over instances.

Noise model	100% of pair terms (132 gates)	50% (66 gates)	25% (32 gates)
No error	69.7%	68.3%	62.3%
IonQ Aria 1 rates	45.2%	**52.1%**	52.8%
IonQ Forte 1 rates (model replaced Sept. 2025)	60.5%	61.9%	58.9%

With half the gates, the sparse layer loses 1.4 points without noise and gains 6.9 points under the Aria 1 rates. Under the older Forte 1 rates the gain is 1.4 points.

Hosted runs. We ran `qollab/main.py` on Qollab: one instance, 12 repairs, depth 2, 2,000 shots, one run per backend. The gain over random plans is the random mean (73.788) minus the shot mean. The noiseless gain with every pair term is 4.569.

Backend	Pair terms	Two-qubit gates after compiling	Mean cost	Gain over random	Share of the noiseless gain	Best-plan shots
Built-in simulator	100%	132	69.219	4.569	100%	0.70%
Built-in simulator	50%	66	69.281	4.507	98.6%	1.10%
IonQ Forte 1 noise model	100%	132	71.326	2.462	53.9%	0.10%
IonQ Forte 1 noise model	50%	66	70.390	3.398	**74.4%**	0.75%
IBM Fez noise model	100%	741	71.884	1.904	41.7%	0.30%
IBM Fez noise model	50%	357	70.824	2.964	**64.9%**	0.45%

On both noisy backends the sparse circuit keeps more of the gain than the full one. On the IonQ model the circuit compiles to the same number of two-qubit gates as written. On the IBM heavy hex model it grows by a factor of 5.4 to 5.6. These are single runs on one instance.

5. Where classical methods win

We built the classical baselines first: a proven optimum, an exact method for restoration on a tree, best-first search on the fitted cost, local search, thermal sampling, Karmarkar–Karp differencing, and the published heuristic. At these sizes, classical methods are as good as or better than QAOA in every one of the following:

Comparison	Classical	QAOA
Budgeted restoration, 6 of 12 lines, optimum reached (30 instances)	Best-first search on the fitted cost: 30 of 30, 145 evaluations on average	Depth 3: 16 of 30 (depth 1: 5; depth 2: 9)
Same task, exact tree method	Optimum on 30 of 30 with one plan scored	—
Crew assignment, optimum reached at equal budget (30 instances)	Best-first search: 16 of 30	6 to 8 of 30
One urgency level, 14 repairs, hours difference	Karmarkar–Karp: 0.0207	Depth 3, best of 100: 0.0856 (best of 1000: 0.0106)
Same fitted pair cost on a tree, best of 100 (12 repairs)	Thermal sampler at its best temperature: 0.098%; at QAOA's mean cost: 0.147%	Depth 3: 0.134%
QAOA shots as starts for local search (12 repairs)	Random starts: 9.8% reach the best plan	9.0% to 9.3%

The comparison in Section 4.1 is between two quantum encodings. It shows that CrewCut is the better way to put this problem on a quantum computer. It does not show that a quantum computer is the better way to solve it.

6. Limitations

- **Sizes.** The comparison with standard QAOA runs to 10 repairs (20 qubits for the baseline). Beyond that the baseline cannot be simulated exactly with noise. All other results are at 18 repairs or fewer.
- **Two crews.** With three or more crews, "same crew" is no longer a pair term, and the encoding does not carry over unchanged.
- **Noise.** Table 4 uses a documented noise model, not hardware. The hosted runs are single runs on one instance. We make no claim that the ion layout gives better results than a nearest-neighbour layout under noise.
- **Statistics.** The ratios in Table 1 are ratios of medians. The per-instance comparison (30 of 30, 10 of 10) is the stronger statement.
- **Baselines.** The utility rules are implemented from their description in Tan et al. [1], and are checked against that paper's Algorithm 5 and Theorem 5.

7. Conclusion

For two crews, the restoration schedule is a pair cost on one qubit per repair. Encoding it that way, rather than in the standard one-hot form with a penalty, halves the qubits, more than halves the pair gates, removes invalid shots, and returns a best plan 3 to 5 times as often at depth 2. The gap grows with problem size. With a sparse cost layer, it also keeps more of its gain under noise. The best of 100 CrewCut shots beats the published heuristic and the utility dispatch rules on the IEEE 13 node feeder. Exact classical search on the same cost remains stronger at every size we tested.

8. Claims and their sources

Claim	Result file
Table 1, Table 2, per-instance wins	`results/standard_qaoa.txt`, `results/standard_qaoa_10.txt`
Baseline at the random valid-plan rate	`checks/standard_qaoa_checks.csv`
Gate counts, all-pairs and heavy hex	`future/A11_ladder/layout_gate_counts.txt`
Table 3, dispatch rules (300 instances)	`results/dispatch_rules.txt`
Table 3, QAOA and the conversion algorithm (20 instances), proven optimum	`results/crew_assignment.txt`
Equal-budget regret; best-first, greedy and QAOA counts	`results/budget_experiment.txt`
Growth with size and depth; Karmarkar–Karp; local search starts	`results/qaoa_limits.txt`
Thermal sampler	`results/samplers_and_protection_rule.txt`
Exact tree method	`results/tree_optimum.txt`
Table 4	`future/A8_noise_study/sparse_layer_under_noise.txt`
Sparse layer without noise	`future/A11_ladder/sparse_cost_layer.txt`
Hosted runs	`results/qollab/qollab_run_*.txt`

References

1. Tan et al., *Scheduling Post-Disaster Repairs in Electricity Distribution Networks*, https://arxiv.org/abs/1702.08382
2. E. Farhi, J. Goldstone, S. Gutmann, *A Quantum Approximate Optimization Algorithm*, https://arxiv.org/abs/1411.4028
3. A. Lucas, *Ising formulations of many NP problems*, https://arxiv.org/abs/1302.5843
4. M. Skutella, *Convex quadratic and semidefinite programming relaxations in scheduling*, J. ACM 48(2), 2001, https://page.math.tu-berlin.de/~combi/coga-old/publications/techreports/2000/Report-683-2000.html
5. T. F. Rønnow et al., *Defining and detecting quantum speedup*, https://arxiv.org/abs/1401.2910
6. IonQ, *Simulation with Noise Models*, https://docs.ionq.com/guides/simulation-with-noise-models